

해당 코드를 app.py라는 파이썬 파일을 만들어 입력한 뒤 저장해주시면 됩니다

```
In [ ]: # 필요한 패키지 설치
# 아래 명령어를 터미널에서 실행하여 필요한 패키지를 설치합니다.
# pip install streamlit python-dotenv langchain langchain-community langchain-openai duckduckgo-search wikipedia

import os

# Streamlit 라이브러리 임포트
import streamlit as st

# 환경 변수(.env 파일) 로드
from dotenv import load_dotenv

# LangChain 관련 모듈 임포트
from langchain_classic import hub
from langchain_classic.agents import AgentExecutor, create_openai_tools_agent, load_tools
from langchain_classic.memory import ConversationBufferMemory
from langchain_community.callbacks import StreamlitCallbackHandler
from langchain_community.chat_message_histories import StreamlitChatMessageHistory
from langchain_openai import ChatOpenAI

# .env 파일에서 환경 변수를 불러옴 (OPENAI API 키 등을 설정하기 위함)
load_dotenv(override=True)

def create_agent_chain(history):
    """
    LangChain 에이전트(Agent)를 생성하는 함수
    - OpenAI 기반 챗 모델 사용
    - 도구(duckduckgo 검색, 위키피디아 검색) 로드
    - 메모리(대화 기록 저장) 활용
    """

    chat = ChatOpenAI(
        model_name="gpt-4.1-mini", # OpenAI 모델명 설정 (예: gpt-4, gpt-3.5-turbo)
        temperature=0.2, # 모델의 창의성(랜덤성) 조정
    )
```

```

# 사용할 도구(duckduckgo 검색 및 위키백과 검색) 로드
tools = load_tools(["ddg-search", "wikipedia"])

# 프롬프트 템플릿을 LangChain Hub에서 가져옴
prompt = hub.pull("hwchase17/openai-tools-agent")

# 대화 기록을 저장하는 메모리 설정
memory = ConversationBufferMemory(
    chat_memory=history, memory_key="chat_history", return_messages=True
)

# OpenAI 챗 모델과 도구, 프롬프트를 이용해 에이전트 생성
agent = create_openai_tools_agent(chat, tools, prompt)

# 에이전트를 실행하는 객체 생성 (대화 기록 포함)
return AgentExecutor(agent=agent, tools=tools, memory=memory)

# Streamlit 애플리케이션 제목 설정
st.title("langchain-streamlit-app")

# 대화 기록을 저장할 객체 생성
history = StreamlitChatMessageHistory()

# 이전 대화 내용을 화면에 표시
for message in history.messages:
    st.chat_message(message.type).write(message.content)

# 사용자 입력을 받을 입력창 생성
prompt = st.chat_input("What is up?")

if prompt:
    # 사용자 입력을 화면에 표시
    with st.chat_message("user"):
        st.markdown(prompt)

    # AI 응답 처리
    with st.chat_message("assistant"):
        callback = StreamlitCallbackHandler(st.container()) # Streamlit UI 업데이트를 위한 콜백 핸들러
        agent_chain = create_agent_chain(history) # 에이전트 체인 생성
        response = agent_chain.invoke(

```

```
    {"input": prompt},  
    {"callbacks": [callback]},  
)
```

```
# AI의 응답을 화면에 표시  
st.markdown(response["output"])
```

```
# Streamlit 앱 실행 방법  
# 터미널에서 아래 명령어 실행  
# streamlit run <파일명>.py  
# 예: streamlit run app.py
```

만든 뒤 터미널에서 해당 폴더의 위치까지 이동한 뒤 streamlit run app.py 명령어로 streamlit을 실행합니다.